

Research Plan for Enhancing OneCare's Safety Gate & Triage System

Introduction & Objectives

OneCare's current **safety gate** uses a mix of deterministic rules (e.g. lexicon "red flags") and a lightweight acuity classifier to immediately divert potential emergencies ¹. This ensures explainability and fast (<2s) response for urgent cases. Downstream, a triage module assigns priority scores using both NLP risk flags and a learned acuity model ² ³. The goal of this research plan is to explore **modern enhancements** – including **ML/LLM-assisted techniques** – to improve emergency detection and acuity scoring *without* compromising safety, latency, or regulatory compliance. A key focus is defining what **data** we must collect (and how to do so compliantly) to train our own models. We will address:

- **Data requirements** for training/validating tailored emergency detectors, red-flag classifiers, and acuity scoring models (including dataset schema, size, and class balance).
- **Consent and PHI governance** for using clinical narratives in model development (de-identification vs. pseudonymization, retention policies, audit trails).
- **Evaluation metrics** and validation strategies that align with patient safety (e.g. high recall for emergencies, low false negatives, calibration, and latency monitoring).
- **Roles for LLM components** – when it makes sense to introduce large language models (as feature extractors or second-line reviewers) while keeping a deterministic primary decision path for accountability.
- **Deployment guardrails** for a hybrid rules + ML system handling urgent care routing (runtime safety checks, monitoring, fallbacks, and human-in-loop oversight).

Literature Survey: Tech-Assisted Triage & Hybrid Approaches

Academic and industry work on AI-assisted triage shows a trend toward *hybrid systems* that combine interpretable rules with statistical models for improved accuracy. Traditional triage methods (like ESI in emergency departments) rely on human assessment; recent research demonstrates machine learning can outperform these in acuity prediction ⁴. For example, a 2025 study proposed a **Knowledge-enhanced Uncertainty-aware Triage System (KUTS)** that fed patient demographics, vital signs, and chief-complaint text into a prompt-tuned language model, then used an uncertainty-based classifier to predict ED triage levels ⁵ ⁶. By incorporating both structured data (age, gender, 7 vital signs) and free-text symptoms, this approach improved identification of moderate-acuity cases and overall accuracy ⁷ ⁸. This underscores that a richer feature set and advanced NLP can refine triage beyond what simple rules achieve.

Rule-based vs. ML Hybrid: However, even cutting-edge models are typically *augmented with rule-based safety nets*. Many digital triage systems maintain explicit **"red flag" keyword lists** that trigger automatic emergency diversion regardless of model output ¹. This is for good reason: purely statistical models might miss edge cases, whereas curated medical rules ensure known critical presentations (e.g. *"chest pain"*, *"suicidal ideation"*) are never overlooked. Hybrid designs are recommended in literature and practice. They offer the **nuance of ML** (capturing context and subtle cues) together with the **guarantee of hard-coded checks** for obvious emergencies ¹. OneCare's current safeguard embodies this: if

either the lexicon or the model flags an emergency, the patient is diverted with urgent advice ¹. This principle should continue – any ML upgrade should *complement* but not replace the deterministic safeguards, especially for high-risk cases.

LLM-assisted Triage: Recent research has explored large language models in triage tasks, but with mixed findings. **On the positive side**, LLMs have demonstrated the ability to interpret free-text medical descriptions and sometimes match clinician triage decisions. For example, one study reported an LLM could correctly identify the higher-acuity patient in ~89% of paired ED cases ⁹. Another found that LLM-based tools (e.g. fine-tuned **ChatGPT**) showed promise in improving triage accuracy for non-critical emergency patients when fed real clinical conversation transcripts ¹⁰. **However, safety evidence urges caution:** A *JMIR* evaluation noted that GPT-4 had *lower sensitivity for high-risk patients*, leading to under-triage (missed emergencies) compared to human experts ¹¹. Similarly, symptom-checker style AI systems consistently underperform emergency physicians on urgent cases ¹² – they should serve only as decision support, not final decision-makers in emergency triage. These findings highlight that while LLMs can process clinical text impressively, they may not reliably catch every red flag without supervision. **Key takeaway:** LLMs in triage should be introduced in a constrained, assistive capacity (e.g. suggesting features or providing a second opinion) rather than given autonomous authority over emergency decisions ¹¹ ¹². Any LLM component must be rigorously validated and likely paired with an uncertainty estimate or human review step ⁵ ⁸.

Hybrid rule/ML systems in practice: Industry systems for urgent care triage (e.g. telephone nurse triage protocols, NHS 111 symptom checkers) remain largely rule-based for accountability, but there is movement toward “**ML-on-call**” approaches. For instance, some commercial triage platforms use ML classifiers behind the scenes to recommend a triage level, while a rules engine and/or clinician makes the final call – effectively keeping a deterministic primary path with ML as a supporting tool. We found research on *retrieval-augmented LLM triage* for patient portal messages, where the LLM pulls in a knowledge base (e.g. guidelines) to decide if a message describes an emergency ¹³ ¹⁴. Such systems achieved high accuracy in flagging urgent messages, but they still propose having **knowledge graphs and rule-based prompts** constrain the LLM for reliability ¹³. Another study introduced distribution-shift tests for LLM triage models to ensure they remain robust across different patient subpopulations and hospitals ¹⁵ – reflecting the safety requirement of consistent performance. In summary, the literature suggests **augmented intelligence** is the ideal: use modern NLP/LLM methods to enhance signal detection (e.g. parsing nuanced narratives, handling misspellings or colloquial descriptions of symptoms), but **wrap those methods in a layer of rules, checks, and human oversight** to guarantee safety ¹² ¹¹.

Data Requirements & Dataset Schema for Model Training

To build improved emergency classifiers and triage scoring models tailored to OneCare’s intake, we first need to **curate a labeled dataset** from our operational data. Currently, raw logs/audit events capture some information (safety gate outcomes, clinician follow-up, etc.), but these must be normalized into a structured dataset for ML. We recommend each data sample (each patient intake submission) be represented with at least the following fields:

- **Patient Free-Text Narrative:** The chief complaint or symptom description text provided by the patient (or transcribed from a call). This is the primary input for NLP. It may require preprocessing (e.g. lowercasing, spelling correction, removal of personally identifying info – see PHI handling below). We should preserve **all clinically relevant text** here, as subtle phrasing can impact urgency.

- **Structured Patient Context:** Key attributes about the patient and context of the request. This includes patient **age**, **biological sex** (if available and relevant), any recorded **comorbid conditions** or risk factors (e.g. “diabetic”, “history of heart disease”), and **vitals** if present. In our intake, vitals are optional, but if a patient reports something like blood pressure, heart rate, or temperature, those should be captured as numeric features. Age and vitals often strongly influence triage decisions in clinical settings ⁶. We should also include **time/context** features such as timestamp or hour of submission (to possibly capture out-of-hours context) and **channel/locale** (e.g. “phone-english” vs “web-spanish”) if that might affect how narratives are written or interpreted.
- **Safety Gate Signals:** The outputs or flags generated by the current safety gate for that case. For instance: a boolean or list for **red-flag keywords hit** (e.g. “shortness of breath” true/false), the **emergency classifier score** (if the traditional model produced a probability or class), and the **initial acuity score** assigned (if applicable). Capturing these as features in the dataset serves two purposes: (1) They can be used as input features for new models (e.g. the presence of a red-flag word could be an input feature to a learned model, providing a prior signal), and (2) they become part of the model training labels for retrospective analysis (e.g. we may train a model to mimic or improve upon the current rule+classifier outcome).
- **Final Disposition Label:** This is critical as the **ground truth label** for emergency detection. We propose using a field indicating whether the case was ultimately **DIVERTED** to emergency care or **SAFE_TO_CONTINUE** with routine triage. In practice, this can be derived from audit logs (“urgent_diversion” events vs. normal flow) ¹. To refine labels, we should also incorporate **clinician override notes**: if a clinician later reviewed the case and marked that an emergency was missed or a false alarm occurred, that information should update the ground truth. For example, if the safety gate let a case through but a clinician said “patient actually needed emergency care – sent to ER,” we should label that narrative as an emergency that was initially missed. Conversely, if a diversion happened but a clinician notes “false alarm – non-emergency,” we might label that as non-emergency in truth (depending on how we want to train the model – we may choose to still label it as emergency for a conservative approach). Clinician override notes can thus refine our labels for both **false negatives** (missed emergent cases) and **false positives** (over-triaged cases). We should capture a text or coded field for any **override reason** or outcome (e.g. “clinician confirmed emergency” or “patient sent to A&E next day”).
- **Acuity/Priority Outcome:** Aside from the binary emergency diversion, we should label the *priority level* assigned to the case in triage (e.g. *stat*, *urgent*, *routine*). If our acuity scoring model is being improved, having the final triage priority (especially if validated by clinicians or by patient outcome) as a label will allow training a model to predict priority. Note that priority is a multi-class label (perhaps 3-4 levels). We may derive it from the created task or appointment (e.g. whether it was marked as needing contact within 2h vs 48h, etc.). If clinician later adjusted the priority (overrode an AI suggestion in triage), that feedback should be recorded as well for model training (learning from those corrections).

Granularity & Volume: Each intake submission represents one labeled instance. We need a sufficiently **large and balanced** dataset to train robust models. Emergencies (true urgent diversions) are typically rare – let’s say (for illustration) ~2-5% of all intakes result in diversion. That class imbalance means we should plan for techniques like *stratified sampling*, *oversampling of positive cases*, or *cost-sensitive learning* to ensure the model learns the emergency class well. In terms of volume, training a reliable classifier might require on the order of **thousands of positive (emergency) examples**. For instance, if we have 50,000 past intakes with ~5% emergencies, that yields ~2,500 emergency cases – likely a reasonable starting dataset. More is always better, especially to capture the variety of ways emergencies can be described. **Data augmentation** could help (e.g. using synonyms or paraphrasing narratives, or adding small perturbations), but since patient safety is on the line, real examples with clinician-validated outcomes are gold standard. We will likely want to continuously expand this dataset as more labeled cases accumulate (with clinician review).

Open-Access Datasets & Benchmarks: We will supplement our internal data with any external datasets that resemble our task, either for model pre-training or benchmarking. Notable open resources include:

- **MIMIC-IV ED:** The MIMIC database (from Beth Israel Deaconess/PhysioNet) includes de-identified emergency department records for hundreds of thousands of visits. Notably, it has triage information such as vital signs and **chief complaint text** for ED patients ¹⁶, along with the triage acuity level assigned. This could serve as a pre-training corpus for our acuity model – e.g. training a model to predict ED triage level from chief complaint + vitals. **Licensing/Usage:** MIMIC is open-access for research but requires signing a data use agreement (forbidding re-identification). It can be used to pre-train or benchmark, but since it's US hospital data, any model should be fine-tuned on our own (UK urgent care) data for specificity. Direct use in production might be limited due to domain differences and the need to ensure no hidden biases (plus MIMIC is for academic use; commercial use would need review of the license).
- **Triage challenge datasets:** The research community has released some datasets from triage studies. For example, one 2023 study on AI triage used **166,175 patient cases from two hospitals** to train an AI chatbot for triage (likely not open-access, but the *Journal of Emergency Medicine* paper ⁹ suggests the scale of data needed). Another resource is the **Manchester Triage System (MTS) dialogue dataset**, which contains ~1,700 short doctor-patient conversation summaries with associated triage outcomes ¹⁷ ¹⁸. This MTS-Dialog set (if available openly) could be useful for benchmarking an LLM's ability to triage from conversational data, although it might be more geared to emergency department context.
- **Symptom checker logs:** Some companies or research projects have released de-identified logs of symptom checker interactions (e.g. the C-PATH project combined medical question-answer datasets and doctor-patient dialogues for training an LLM ¹⁹ ²⁰). These could provide synthetic training examples for how patients describe symptoms and what triage advice was given. We must check licensing – often these are research-only. They can be used for **benchmarking** our model's performance on a common task (to see if our model is on par with published results), but for **pre-training** or fine-tuning, we should ensure we have rights to use them. Many open datasets in healthcare (like MIMIC) permit internal research/quality improvement, but if we were to incorporate any data into a commercial product, legal review is needed. We will thus primarily use open data for model evaluation and possibly as **additional unlabeled data** to pre-train language models on “clinical text”.

Data Schema Recommendations: Our data schema should align with the features our current system uses, while adding any new ones needed for richer modeling. OneCare already extracts features like symptom keywords, age, comorbidities, and vitals. We will retain those and ensure the dataset schema has fields for each. Additional schema elements we propose:

- **Symptom/Urgency Annotations:** We may want to label certain narratives with which **symptom categories** or **red-flag categories** they contain. E.g., a narrative might be annotated as containing a “*chest pain*” flag, a “*neuro symptom*” flag (like weakness on one side), etc. These could be derived from our lexicon or an NLP model. Capturing them explicitly in the dataset can help ensure class balance across different emergency types (we'd know if we have at least e.g. 100 chest-pain cases, 100 shortness-of-breath cases, etc.). This ties into the **class balance** aspect – we need not only overall emergency vs non-emergency balance, but also coverage of the variety of emergencies. This helps a model not overfit to one kind of emergency.
- **Numeric Score Fields:** If our current acuity model outputs a continuous score or probability, include that as a feature/label. Same for any triage priority score. This allows training regression models or calibrating thresholds. We should also add a field for **threshold results** (e.g. was the

acuity score above 0.9 = “stat” priority?). Essentially, the dataset can have both the raw score and the thresholded category.

- **Text Embeddings (optional):** During model development, we might enrich the dataset with precomputed text embeddings from a medical language model (like BioClinicalBERT or similar). These aren't *stored* in final production, but as a tooling step: e.g., we could embed each narrative with a pre-trained model to cluster or visualize the data, helping identify if emergent cases cluster by language features. This can guide feature engineering (for instance, if many emergencies mention certain concepts, we ensure our lexicon covers them).

Overall, our dataset should be structured, **auditable**, and linkable (under consent) to outcomes. Each entry ideally has a unique ID to join with our audit logs or patient records if needed (under strict access controls), so that we can later add outcomes (like if a patient actually was hospitalized, etc. – though that may be beyond our current scope, it's something to consider for future model validation). By normalizing these records, we set the stage for training supervised models for **emergency detection (binary classification)**, **red-flag identification (multi-label classification or sequence tagging)**, and **acuity/priority scoring (multi-class classification/regression)**.

PHI Handling, Consent & Privacy Compliance

Using patient narratives and clinical data for model training brings significant **privacy and regulatory obligations**. We must capture and use this data in a way that respects patient consent, ensures confidentiality (no unauthorized access or sharing), and complies with laws like HIPAA, GDPR, and UK regulations (NHS policies, Data Protection Act). Below we outline a plan for **governing consented PHI** in model development, including de-identification strategies, data retention, and audit requirements.

Consent and Purpose Limitation: Under UK NHS guidance, using patient data to train AI models is considered a “**purpose beyond individual care**” ²¹. In other words, when we take a patient's narrative and use it to improve an algorithm for future patients, we are no longer using the data *to treat that same patient*, but for a secondary purpose (quality improvement / research). NHS policy and GDPR both require that patients be informed of such use and, in many cases, give consent or have the opportunity to opt out ²². OneCare should ensure its **Privacy Notice** explicitly covers the use of intake data for service improvement and model training. Ideally, we obtain at least an “**implied consent**” for using data in quality improvement (for example, NHS patients implicitly allow their data to be used in audit and service development as long as privacy is protected ²³ ²¹). For more sensitive uses, or if data is not fully anonymized, we might need either *explicit consent* or **approval under regulation**. In the UK, if we needed to use identifiable patient info for model training without consent, an application to the **Confidentiality Advisory Group (CAG)** under Section 251 would be required ²⁴ – this is a governance body that can authorize such use if it's in the public interest. Our plan will aim to avoid that by **de-identifying or pseudonymizing data to the highest extent possible**.

De-identification vs. Pseudonymization: We will implement a **de-identification pipeline** to strip out personal identifiers from the narratives and dataset before data scientists access it for model training. De-identification can be done via:

- *Rule-based stripping:* Using patterns and dictionaries to remove obvious identifiers (names, phone numbers, NHS numbers, addresses, etc.). For example, we can regex for sequences of digits that look like phone or NHS ID, remove email addresses, redact any word that matches a list of common first/last names, etc. This approach is straightforward and transparent, but can miss context-sensitive cases and requires maintenance for edge cases ²⁵. For instance, a rule-based system might mistakenly remove “Parkinson” thinking it's a name, or fail to remove “Dr. Smith” if not coded properly ²⁶.
- *NER-driven masking:* Using a **trained NLP model** to identify PHI entities (Person, Location, Hospital, Date, etc.) in text. Machine learning models (like a BiLSTM-CRF or Transformer trained on de-

identification tasks) can catch more variations and use context to avoid false positives ²⁷. Studies show ML-based de-id generally improves recall and robustness to diverse text ²⁵. We can leverage existing de-id models (e.g. models from the **i2b2 de-identification challenge** or **Physionet** tools). A recent evaluation even explored GPT-based de-identification, though combining ML with rule post-processing was noted to add complexity without huge gains ²⁸.

- **Hybrid workflow:** Our recommendation is a **hybrid de-identification pipeline**: run a rule-based scrubber and an ML model, and combine their outputs. In practice, we can first apply conservative regex rules (to catch straightforward identifiers with near-zero false positives), then apply an ML model to catch things the rules missed (like a rare surname, or “my daughter Jane”). The ML output can be reviewed or validated on samples to ensure it’s not erroneously removing medical terms. Hybrid approaches have achieved the best performance in de-id competitions ²⁹, though they require careful engineering (and possibly an adjudication step if the rule and ML outputs conflict).

Tooling: Since we prefer OSS, we can consider tools like **Philter** (open-source PHI masker), **pyDeid** ³⁰, or **spaCy** with a medical NER model (e.g. the *medSpaCy* project has components for detecting patient names, etc.). These can be run on-premises so no data leaves our environment. If we evaluate vendor solutions: for example, AWS Comprehend Medical or Google Healthcare API have de-identification services. They tend to be high-quality but would involve sending data to cloud providers – which means we’d need Business Associate Agreements (HIPAA) and to ensure GDPR compliance (likely these are not EU-hosted). Given our constraints, an in-house pipeline is preferable. If we did use a vendor tool, we’d note that PHI is leaving our system and ensure encryption and deletion policies are in place (and factor in integration effort).

Re-identification and Linkage: We plan to *retain a linkage* to outcomes under consent, meaning we cannot fully anonymize (which would break the ability to tie back to patient or follow up on outcome). Instead, we will **pseudonymize**: replace patient identifiers with codes. For example, we assign a random ID to each patient in the dataset, and keep a secure mapping file accessible only to a few authorized staff. This way, the training dataset contains no directly identifying info (names, contacts removed; ID is random), but if later we need to audit or add outcome data (like if a case resulted in hospitalization), we can, under controlled access, link it. This approach aligns with GDPR’s concept of pseudonymization – data is still considered personal data (since the key exists), but risk is reduced and it’s acceptable under *legitimate interest* or *research* provisions with safeguards. We must store the re-identification key securely (maybe in our audit ledger or a consent management system) and log any access to it.

Retention and Minimization: We will apply **data minimization** principles ³¹ ³² – use only the data necessary for the modeling purpose. For instance, we likely do *not* need patient names, exact addresses, or full DOB for triage models, so those are removed. We might not even need the patient’s real identity at all, just their clinical features. We should also exclude any data fields that aren’t relevant (e.g. if the intake form had an attachment image, we would not include that in text model training unless we plan a vision component). Regarding **retention**, OneCare’s config suggests a long retention (maybe 10 years) for audit ³³, but for model training data, we might not need to keep every narrative indefinitely. A practical approach: maintain a moving window (say the last **X** years of data) for model training, and archive older data separately. The model lifecycle pseudocode in our internal design calls for training data from a *6-12 month window* ³⁴ – which is a good default for keeping models current. We can retain longer for audit, but limit the active training dataset to recent, relevant cases, and of course exclude patients who opted out of data use.

Audit and Access Controls: Every use of patient data for model training should be **audited**. We will log when data is accessed, by whom, and for what purpose (similar to how we audit clinical accesses). Only authorized staff (data scientists, ML engineers under appropriate confidentiality agreements and

training) should have access to the de-identified corpus ³⁵. We will enforce **role-based access** – e.g., an engineer can access aggregated training data but not the key to re-identify patients ³⁶. If we use any external collaborators or vendors for model development, they must sign a Data Processing Agreement/BAA and meet our security standards (encrypted storage, no secondary use, etc.) ³⁷ ³⁸.

HIPAA Considerations: In the U.S. context (if any data is from there or we apply HIPAA-like rules), using PHI for developing AI could be deemed **healthcare operations**, which is permitted under HIPAA's Privacy Rule without individual authorization – provided it's for internal quality improvement and not external research ³⁹. However, there is some ambiguity: a recent HIPAA commentary noted that general "training AI technology" might *not* be clearly listed under treatment, payment, or operations (TPO) ⁴⁰. To err on the safe side, we treat it with research-like safeguards. If not TPO, we'd theoretically need either patient authorizations or de-identification. Thus, our plan to de-identify/pseudonymize plus use data under the umbrella of service improvement with no external disclosure should keep us in compliance. We'll also apply **minimum necessary standard** – only use PHI fields truly needed for the models ⁴¹ ⁴². For example, we won't feed the model the patient's exact address or full name (which it doesn't need to decide urgency). This also protects against "data overreach" where AI might inadvertently learn from irrelevant PHI ⁴³.

GDPR and UK Compliance: GDPR requires a lawful basis for processing. Possible bases here: **"Provision of health care (medical diagnosis/treatment)"** could be argued if the model directly contributes to care (but training phase is indirect). More fitting might be **"legitimate interests"** of the company to improve its service, or if we classify it as research, **"task in public interest"** for improving healthcare. We must also honor patient rights: if a patient says *"don't use my data for anything except my care"*, we need to respect that (opt them out of the training dataset). The NHS AI guidance emphasizes transparency and the ability for patients to object ⁴⁴ ⁴⁵. We will provide a mechanism for patients to withdraw consent for data usage beyond their care. Additionally, any model training on UK patient data likely requires a **Data Protection Impact Assessment (DPIA)**. We should conduct a DPIA that outlines the data flow, risks, and mitigations (like de-identification, access control, etc.) and have it reviewed by our Caldicott Guardian or equivalent (as recommended: involve the DPO and Caldicott Guardian early ⁴⁵).

Special Considerations (Legal Review Needed): Despite all these measures, there are areas that will need **additional legal review**: for instance, if we decide to use a third-party service or cloud for any part of model training (even something like using cloud GPUs with our data), we must ensure data residency and contractual safeguards. If we ever consider sharing a de-identified dataset (say with a university for external model testing), that crosses into research territory requiring ethics approval and perhaps patient consent. Also, if our model's outputs will be used in a regulated manner (which in healthcare they likely will), we must comply with standards like **DCB0129/0160** for clinical risk management ⁴⁶ – essentially documenting hazards and mitigations (which include data handling hazards). We note that even a perfectly de-identified dataset can carry re-identification risk if combined with other data; we should have our de-identification checked by an expert ("expert determination" under HIPAA) if possible ⁴⁷. Finally, we have to consider the risk of the model itself memorizing and potentially regurgitating PHI. Large models have been shown to sometimes output training data verbatim, posing a privacy leak. To counter this, we should test the trained model for such behavior (e.g. prompt it with partial inputs from training cases to see if it outputs names) ⁴⁸. If we find any risk, either further restrict model capacity or implement filters on model outputs. All these points underscore that we will work closely with legal and compliance teams at each step.

In summary, our data governance will ensure that **only de-identified (or heavily pseudonymized) clinical narratives and related data are used for model training**, with strict consent management, logging, and compliance checks (HIPAA's minimum necessary, GDPR's data minimization, NHS's

transparency and DPIA processes). These safeguards will enable us to leverage real-world clinical text to train better models while upholding patient trust and meeting regulatory obligations ²⁴ ⁴⁰ .

Evaluation Metrics and Validation Strategy for Safety

Designing **evaluation metrics** and validation protocols is crucial, as our models will be making potentially high-stakes decisions (emergency diversion). We will adopt metrics that emphasize **patient safety**, especially avoiding false negatives (missed emergencies), and strategies that validate model performance both offline (historical test data) and online (shadow mode or pilot deployment) before any full release.

Key Metrics: For emergency detection (binary classification of “divert vs safe”), **Recall (Sensitivity)** on the positive class (emergency) is paramount. A false negative (failing to divert a true emergency) is the worst outcome, so we aim for recall as close to 100% as possible. We will measure recall at the operating threshold we choose. We’ll also monitor **Precision** (positive predictive value) because too many false alarms (low precision) can overwhelm staff and erode trust. In practice, we may fix a recall target (e.g. ≥ 0.99) and then evaluate what precision that yields. A high recall, moderate precision model might be acceptable initially (since the current lexicon probably has false positives built in for safety). We can also use **F1-score** for a single summary metric, but given class imbalance, we might favor **F2-score** (which weights recall higher) to reflect our priorities.

For acuity scoring (multi-class priority), we will use metrics like **accuracy** and **weighted F1**, but also **calibration** and **confusion matrix analysis**. Calibration is important because if the model outputs a risk probability, we want that probability to correspond to true likelihood of needing urgent care. We can use **Brier score** or calibration curves to ensure the model’s score can be interpreted meaningfully (e.g. if it says 5% emergency risk, is it truly 5% in reality?). **ROC-AUC** and **PR-AUC** will be measured on the emergency classification to judge general discrimination ability independent of threshold. However, we care less about AUC and more about performance at the extreme high-sensitivity region of the curve. We might report the **Precision-Recall curve** and ensure the area in the high-recall range is good.

Another critical metric is **False Negative Rate (FNR)** on emergencies (which is $1 - \text{recall}$). We want FNR as close to 0 as feasible. We will likely set an upper bound (e.g. “no more than 1 in 100 emergencies can be missed” or even stricter). If our test data has zero false negatives, that’s ideal, but with limited data we might estimate a confidence interval. We can also compute **Negative Predictive Value (NPV)**, which is relevant if the prevalence of emergencies is low – NPV will usually be very high, but any dip could signal concern.

For general triage priority, we might evaluate **weighted error rates**: misclassifying an emergency as routine is far worse than routine as emergency. We can assign a cost matrix and calculate a **cost-weighted error**. For example, a false negative emergency might be assigned a weight of 50, a false positive maybe 1 or 2, etc., to quantify overall risk.

Latency and Throughput: Since latency is a requirement ($p_{95} < 2s$ for triage decisions ⁴⁹), we will include **response time** as a metric in evaluation. Offline, we ensure model inference time on a single case is well below the threshold (with some headroom for integration overhead). We can measure the average and 95th percentile inference times on a batch of test inputs. If an approach is too slow (e.g. a very large model might take $>2s$), it’s not viable without optimization. We’ll incorporate latency into our model selection criteria. For instance, if a simpler model (like logistic regression on embedding features) achieves slightly lower accuracy than a huge transformer but is 10x faster and more predictable, we might favor it for now due to the hard latency limit.

Offline Validation Procedure: We will split our curated dataset into training and holdout sets (and possibly a dev set). We'll use stratified sampling to ensure the emergency cases are proportionally present. We will perform cross-validation for robust estimates if data volume is moderate. During development, we'll do error analyses on false negatives and false positives to refine features and thresholds. Once we have a candidate model, we will test it on:

- **In-domain holdout:** a test set from the same distribution (e.g. past 6 months data we set aside). This gives baseline performance metrics (recall, precision, etc. as above).
- **Time-based split:** It's often wise in healthcare to train on older data and test on newer data to see if performance degrades over time or if there's concept drift (e.g. new ways patients describe things, or a seasonal change in complaints). So we might use, say, last year's data to train and then test on more recent cases. This mimics prospective performance.
- **Out-of-distribution scenarios:** We will compile some *stress-test cases* – e.g. rare but critical scenarios gleaned from medical knowledge or simulation. For example, we can craft a few examples of obviously life-threatening narratives (like “crushing chest pain, sweating, pain in left arm”) and make sure the model flags them. If we have access to any external benchmark (like the MTS-dialog or others), we can run our model on those to see how it fares compared to known results. Ensuring the model doesn't overfit to our phrasing and can handle generally any emergency description is important.
- **Fairness checks:** We will evaluate model performance across subsets, e.g. by patient age group, gender, possibly by language/locale if we support multiple languages, and by any other relevant group (if data allows). The aim is to detect if the model is less sensitive for certain groups. For example, some studies find algorithms perform differently for different demographics; we must ensure, say, that female vs male narratives are equally recognized for, say, chest pain (since heart attack symptoms can be described differently). This fairness monitoring is also highlighted in our internal governance ³⁵. If discrepancies are found, we might adjust the model or add specific rules to compensate.

Online Validation & Pilot (Shadow Mode): After offline tests indicate the model is safe, the next step is **shadow deployment**. This means running the new model in parallel with the existing system, without affecting actual triage decisions. We will deploy the model to receive each intake (in real-time or via a replay) and record what *it* would have done (divert or not, priority score, etc.), while the current rules continue to operate as normal. Over a period (e.g. several weeks), we collect comparisons between model decisions and actual outcomes. Key things to track:

- **Misses by the model:** Any case where the model said “safe” but the patient was actually diverted by our rules or later found to need emergency care. Even one such case is a red flag. We'd review those narratives to see why the model failed – if it's a systematic blind spot, we address it (feature engineering or adding that scenario to training data).
- **Extra catches by the model:** Cases where our current system did *not* divert, but the new model would have. Each of these is essentially a potential false positive of the model *or* a false negative of our current system. We have clinicians review these. If many of them turn out to be legitimate emergencies that our rules missed, that's a huge win (it means the model is catching things earlier). We then have evidence to adjust the system. If they are mostly false alarms (model overreacting), we tune the model or threshold to be less sensitive to those signals.
- **Aggregate metrics in shadow:** We'll compute the model's precision/recall in the live setting as well, comparing to ground truth outcomes (using clinician override and follow-up data as truth). We also monitor **model confidence calibration**: when it says “90% emergency”, was it right 9 out of 10 times? This can inform how we set thresholds.
- **Latency and robustness in production:** We measure how fast the model responds in the real environment. Also, we see if any technical issues arise (memory, errors). This is part of validating that the model can run reliably under load.

We might start shadow mode on a subset of traffic (e.g. 10% of practices or a certain region) to limit risk and data handling, then expand. This is akin to a **canary deployment** approach. In fact, our internal

MLOps plan suggests rolling out a new model to ~5% and monitoring ³⁴. We will follow that: deploy the model to a small percentage of cases (with the old system as backup decision-maker), and confirm there are no adverse events or major discrepancies before scaling up.

Safety Validation: Before any full deployment, we'll conduct a "**safety case**" evaluation (per DCB0129). This involves verifying that for a defined set of hazards, the mitigations (rules or model behavior) are in place. For example, one hazard: "Model fails to identify a cardiac arrest symptom description -> patient not diverted -> harm." Mitigation: Red-flag rule on "chest pain" still catches it; also model was tested on similar phrases. We will simulate a list of critical scenarios (maybe using clinician input: "these 20 narratives must always be diverted"). The model must divert all of them. If any are missed, we cannot deploy until fixed. This is essentially unit testing the model on safety cases.

Ongoing Monitoring Metrics: Once a model is live, we will monitor in real-time: - **Emergency rate:** How many submissions are being flagged emergency by the model vs historically? If this drops unexpectedly, that's concerning (could mean model is under-triaging). We might set an alert if emergency rate falls below a floor unless there's a known reason.

- **Override rate:** If clinicians frequently override the model's decisions (e.g. they keep finding emergencies that model missed, or they note many false positives), that feedback loop should trigger retraining or rule adjustments. We will capture override events as structured data (perhaps feeding them back into the next training iteration – outcome-based learning ⁵⁰).

- **Calibration and drift:** We log model prediction scores and periodically assess if they're drifting (e.g. over time, is the model becoming more or less confident?). Data drift (change in input distribution, say a new slang term for a symptom becomes common) can degrade performance. We plan to retrain or fine-tune the model regularly (maybe every 6-12 months as data accumulates) to keep it current ³⁴. Monitoring input features (like distribution of common words in narratives) might give early warning of drift. - **Latency and uptime:** We will track the model service's latency in production (must remain under threshold) and failure rates. Any time it fails or is too slow, the system should automatically fallback to rules (we will implement this as part of guardrails).

Validation of Acuity Scoring: For the non-binary priority assignment, we can validate by checking outcomes like: Did urgent cases actually get seen sooner? (This might be longer term outcome validation). Or compare model's priority vs clinician's assessment. We could do a small study where clinicians blindly rate the priority of a sample of cases and see if the model agrees significantly. If disagreements occur, evaluate who was "right" by outcome or consensus. Additionally, **calibration of priority:** if the model says X% stat, does roughly that percentage of patients truly require immediate attention? We want to ensure, for example, that among those labeled routine, none had an adverse outcome from waiting. This could tie into a longer-term **prospective validation:** after deployment, track if any patient initially triaged as routine was later found to need emergency care or had to be escalated urgently – that would indicate a miss in scoring. Those instances would be investigated and form new test cases.

Summary of Metrics:

- Emergency classifier: Recall (target ≥ 0.99), Precision, F1/F2, FNR (target ~0), NPV (~1), PR curve.
- Acuity model: Accuracy of class, weighted error (penalize under-triage), calibration (e.g. calibration slope), perhaps **ECE** (expected calibration error).
- Latency: 95th percentile under 2s.
- Safety tests: 0 critical scenarios failed.
- Online: % agreement with current system (initially), override counts, incident count (ideally 0 incidents where patient harmed due to model decision).
- We will also track **False Positive Rate** because false positives do burden staff – we might have a tolerance threshold like "no more than X% of total cases are unnecessary emergency diversions" such

that we maintain service levels. This could be balanced by adjusting model threshold if needed. For calibration, we want high-risk scores to correlate with actual emergencies – measuring **AUROC** helps ensure general rank ordering is good, but our focus is the extreme end of the distribution (the tail of very sick vs rest).

By employing these comprehensive metrics and a phased validation (offline → shadow → limited release → full release), we align with the high safety standards required in urgent-care routing. Our strategy is to prove the model's safety and effectiveness in parallel with the existing system before it ever takes over primary decision-making ³⁴. Only after it consistently meets or exceeds the current safety gate performance (especially on not missing emergencies) will we consider it for primary use, and even then with guardrails as discussed next.

Role of LLM Components & Deterministic Pathways

Modern LLMs offer powerful text understanding, but deploying them in our triage flow must be done judiciously. We propose introducing LLM-based components **incrementally**, in roles that **assist or augment** the deterministic logic rather than replace it. The guiding principle is: *the primary emergency decision path remains rules-driven and transparent, while LLMs operate on the side – adding insights or double-checking – under strict oversight*. Here are scenarios where using an LLM (or other advanced NLP) makes sense and how to do it safely:

- **Feature Extraction from Narratives:** An LLM (particularly a domain-tuned one) can parse the free-text narrative and extract structured features that our current rules/model might miss. For example, a patient might describe “it feels like an elephant is sitting on my chest.” Our keyword list might not catch “elephant” as a proxy for severe chest pressure, but an LLM could interpret that as chest pain and shortness of breath. We can deploy an LLM-based **entity/tag extractor** that reads the narrative and outputs a set of findings: e.g. {"symptoms": ["chest pain", "dyspnea"], "duration": "2 hours", "context": "at rest"}. This transforms unstructured text to structured data. Critically, this extraction can happen *behind the scenes*: the LLM's output feeds into our existing rule engine. For instance, if the LLM labels “chest pain”, our red-flag rule for chest pain triggers the diversion as usual. Here, the LLM acts as a smarter parser, expanding our lexicon coverage. Because the final decision is still taken by the rule (which is explainable: “keyword chest pain present”), we haven't compromised determinism. We'd need to validate the LLM's extraction quality (perhaps start with it in shadow mode comparing to our simpler keyword approach). If it consistently finds relevant symptoms and maybe even fewer false positives (e.g. it knows “Parkinson's” is a disease, not a name), it can be a valuable assistant. Essentially, this is using LLM **as a feature engineer** for our traditional models. Some research supports this approach: constructing prompts that feed patient info into an LLM to get enriched representations improved triage modeling ⁵. As long as we treat the LLM output as just another input feature (with careful testing), the critical safety gate can remain rule-based or a small model making the ultimate call.
- **Secondary Triage Review (“LLM on call”):** We could employ an LLM as a *second pair of eyes* on cases that are borderline. For example, if our primary system deems a case **SAFE to continue** (no red flags triggered, model score below emergency threshold), we might have an LLM do a sanity check. We could prompt it with something like: *“The following is a patient's complaint. The automated triage system did not find it emergent. Do you see any indication of a life-threatening emergency in this narrative? Answer yes or no and explain if yes.”* If the LLM responds with concern (e.g. “Yes – the patient mentioned sudden weakness in arm which could be stroke”), we do not automatically divert the patient, but we *flag the case for human review*. Essentially, the LLM can

raise an alarm that the primary system might have missed. This is a form of **LLM-based oversight**. It's important to set this up such that the LLM's output does not directly trigger patient-facing actions, but rather goes to a clinician or a higher-tier rule. For instance, we could implement a rule: *If LLM says "emergency likely" AND primary says "safe", then escalate to a nurse to manually review immediately*. This way, the deterministic logic is still in control (the rule to escalate is predetermined), and the LLM is a consultant. We would thoroughly test this mechanism on historical cases to ensure the LLM doesn't frequently cry wolf or miss obvious ones. If tuned well, it could add an extra safety net. This addresses the question of *when* to introduce LLMs: doing so **after** the primary decision, as a form of QA, is safer than using it as the first filter. Many in the field advocate LLMs as "*second opinion*" tools rather than primary for exactly this reason ¹².

- **Complex Priority and Routing Suggestions:** Beyond emergency vs not, LLMs might help in more nuanced prioritization. For example, the LLM could analyze the narrative and suggest a triage category or even a differential diagnosis list. We might integrate this in the non-urgent flow: e.g., the **scribe service** or a similar LLM could generate a brief summary or "risk factors list" for the clinician. This doesn't affect automated routing, but it can help clinicians spot if something was under-prioritized. Introduce such uses only after proving the LLM's output quality (we already use LLM for summary in scribe, so this is an adjacent extension).
- **LLM for Rare Case Knowledge:** One advantage of large medical LMs (like Med-PaLM or others) is their breadth of training data – they might recognize signs of a rare condition that our current system doesn't know about. We can harness that by letting the LLM incorporate external knowledge. For example, an LLM with retrieval (like feeding it relevant snippets of medical guidelines based on keywords) could potentially flag a case of, say, ectopic pregnancy risk if a patient says "severe lower abdominal pain and dizziness and positive pregnancy test" – even if we didn't hard-code that combination as a red flag. Research on retrieval-augmented LLM triage shows promise in this area ¹³. However, deploying this in real-time is challenging (time and complexity). A pragmatic approach could be to use LLM offline to **generate new rules**: e.g., run through a large corpus of past narratives and have an LLM classify them or explain them, and find patterns in cases clinicians marked as emergencies. If the LLM consistently notes something (like "the patient's described symptom X which could indicate Y"), we might codify that insight into our lexicon or model features. Thus, LLM aids in *feature discovery*. This keeps the runtime path unchanged (still lexicon/model), but improves it over time with LLM-learned insights.

Safety and Timing of Introduction: We should introduce LLM components only **after** establishing robust deterministic baselines. For instance, once our improved ML model is in place and stable, and we have strong metrics, then we can experiment with an LLM secondary check in a limited setting. An LLM is particularly useful for **language understanding** improvements – e.g., handling multilingual narratives (if we ever intake non-English) or parsing colloquial language. If we find our model struggling with those, an LLM fine-tuned on multi-language medical text might help. The key is to **keep LLM usage interpretable and constrained**. We should log LLM outputs and have traceability (e.g. if it influenced a decision, we record its "opinion"). Also, any time an LLM is used, we maintain the policy of human confirmation for critical outcomes ⁴⁶. That is, if an LLM actually suggests a clinical action (like "this might be an emergency"), a human or a rule must confirm – which is exactly what our design will do. This approach mirrors the concept of "LLM on call" – the LLM is consulted like a remote specialist, but a primary physician (our rule engine or clinician) still signs off on the decision.

Latency Considerations: Running a large model (like GPT-4 or even a local large model) synchronously in the patient intake flow could violate our $\leq 2s$ requirement. Therefore, any LLM we use in real-time must be optimized or relatively small. We might explore **distilled models** or smaller fine-tuned

transformers (like a ClinicalBERT or DistilGPT on our domain) that can run on our infrastructure quickly. Alternatively, asynchronous use is possible: for example, if something is flagged safe and offered a same-day callback, an LLM could analyze it in the background within a minute and if it raises concern, we could still call the patient back sooner. This introduces complexity, so we'd likely stick to LLMs that can operate sub-2s if in the immediate loop, or use them offline for later improvement cycles.

Evidence of Value: We will keep an eye on emerging research where LLMs are tested with safety in mind. One interesting direction is using LLMs to estimate their own uncertainty or recognize when a case is beyond their knowledge. The KUTS study's uncertainty-based classifier is akin to having the model say "I'm not sure, better have a human do this" ⁵¹ ⁵² . We can replicate that idea: if we do integrate an LLM, configure it to output an "uncertainty" or allow it to say "I don't know." If the LLM is uncertain, we default to the safe route (e.g., treat as potential emergency or have human review).

In conclusion, LLM components should be **introduced gradually** and in supportive roles: first as *off-line analysts* to enrich our rules, then perhaps as *online assistants* under supervision. By keeping the primary triage path rule/ML-based and explainable, we maintain auditability and regulatory compliance (since every decision can be traced to a rule or a tested model output). LLMs can provide a safety net *on top* of this – ironically, an AI safety net for the AI, catching what the first-pass might miss. This two-tier approach (deterministic first tier, probabilistic second tier) can harness LLM strengths while containing their weaknesses. Any deployment of LLM will include robust **guardrails**: for example, limit its output format (so it doesn't go off-topic), don't allow it to override a red-flag (if our rules say emergency, it *is* an emergency; we wouldn't listen to an LLM saying "maybe not" in that direction), and continuously monitor its alerts for false alarm rates. If at any point the LLM component shows instability or unacceptably low specificity, we can remove or retrain it without affecting the core service (because the core is still intact). This aligns with a fail-safe design philosophy: *the system should default to the safer outcome*. If the fancy NLP fails, we still have the baseline rules as fallback ⁵³ .

Deployment Guardrails, Monitoring & Fallbacks for a Mixed System

When deploying a mixed rules + ML system in urgent-care triage, we must embed **guardrails** at multiple levels to ensure reliability and patient safety. Here we outline recommended safeguards, monitoring processes, and fallback behaviors:

1. Safety Gates Remain in Place: As a fundamental rule, always preserve the **pre-triage red-flag gate** as an override layer ⁴⁶ . No matter how sophisticated our new model is, certain keywords or conditions (the red-flag set) should *always* trigger emergency diversion unless a human explicitly overrides. This ensures that even if the ML model misclassifies a classic emergency phrase, the patient will still get diverted. We can expand and tune the red-flag lexicon based on model findings (e.g. add new phrases the model found), but we do not remove existing critical triggers. In practice, our deployment will likely have the pipeline: *LLM/NLP extraction* → *ML model scoring* → *rules check* → *decision*. The **rule check will force-divert if any high-severity trigger is present, regardless of model score**. This one-way override (rule can override model to positive, but not to negative) guarantees a baseline safety floor. It's akin to how aircraft autopilot has hard limits that it cannot go beyond without pilot intervention.

2. Deterministic Fallback on Errors: The system should be designed to **gracefully degrade** to the simpler logic if the ML component fails or is uncertain. For example, if our model service is down or slow, we automatically proceed using the rule-based triage and a template response ⁵³ . This means we'll implement timeouts: if the model doesn't return in e.g. 1.5 seconds, we abort and use rules (meeting the 2s SLA). Similarly, if an LLM component is used and it returns an "I don't know" or some

incoherent output, we ignore its output and rely on the rest. In essence, *availability of the AI should not impact availability of the service*. We saw this in our failure mode list: “LLM unavailable: use raw transcript + rule-based triage.”⁵³ . We will also monitor the model's performance; if it starts behaving oddly (e.g. high error rate), an automated switch could turn it off (feature flag to disable ML and revert fully to rules until issue is resolved). This kind of circuit breaker prevents a malfunctioning model from causing harm.

3. Thresholds and Human-in-the-Loop for Ambiguity: We will likely set two thresholds for the model's emergency score: a high threshold above which we *confidently divert*, and a lower threshold below which we *confidently do not divert*. In between, we can have a gray zone where a **human review is triggered**. For instance, if the model says 99% emergency and no rule triggered, we may automatically divert (with audit note “ML auto-divert”). If the model says 70% and no rule, that might be ambiguous – perhaps flag for a nurse to quickly review the narrative. This way, borderline cases get an extra layer of scrutiny rather than an automated decision. Over time, we can adjust these thresholds based on how accurate the model is in those ranges. The human-in-loop approach is recommended in clinical AI to catch any misclassifications before they impact care⁴⁶ . We'll ensure the UI/workflow supports this: e.g. such cases could appear in a clinician dashboard marked “Review suggested: possible urgent case.” The clinician can then upgrade it to emergency or confirm it as routine. Their feedback gets logged for retraining.

4. Monitoring and Alerts: We will implement **real-time monitoring** for key indicators:

- **Model Agreement with Rules:** How often is the model suggesting a different outcome than the rules? Sudden spikes in disagreement could indicate either model drift or some unusual trend in patient input (e.g. a new slang). We will set up automated monitoring of “disagreement rate.”

- **Emergency Conversion Rate:** Among cases flagged safe by the system, do any end up being later flagged as emergencies by clinicians? We might monitor if, say, within 24 hours of a safe triage, the patient contacts emergency services or the practice overrides to urgent. If any such case happens, that's essentially a missed emergency. Even one is critical to investigate. So we'll have an alert if a patient marked routine is later escalated to emergency; it could indicate a model miss (or a condition that worsened, but we treat it as potentially a miss).

- **False Alarm Rate:** Track how many diversions turned out to be non-emergencies. If our false positive rate is climbing, we might be overloading emergency pathways. We will tune to keep this within acceptable bounds (aligned with clinical judgment – perhaps compare with nurse triage outcomes historically). If false positives are too high, we adjust thresholds or model.

- **Drift Monitoring:** As mentioned, keep an eye on input distribution (we can embed incoming narratives and see if they shift in embedding space away from training data – a sign of drift). Also outcome distribution: if suddenly 10% of patients are flagged emergency where it used to be 3%, either something real changed (maybe a local COVID outbreak increasing urgent respiratory cases) or the model is drifting. In either case, it warrants attention. We might integrate an **anomaly detection** on the audit stream⁵⁴ – e.g., a spike in “urgent_diversion” events triggers an alert to check if everything is okay.

- **Latency and System Health:** Monitor the response times and error rates of the ML services. If latency starts creeping up to unsafe levels, alert ops to investigate (maybe load issue or memory leak). If any component crashes, automatic fallback ensures continuity but devs should be alerted to fix it fast.

We'll use our observability stack (dashboards, likely with SIEM integration as in our design⁵⁵) to track these in real-time.

5. Audit Trails and Explainability: Every decision made by ML should be logged with an explanation for later audit. For example, if a case was diverted due to the ML model (and not an obvious rule), we log the top contributing factors (if our model can provide them, e.g. SHAP values or at least which keywords

influenced it) ⁵⁰. This is important for compliance and for clinician trust – if a clinician asks “why was this patient marked urgent?”, we should provide a rationale: e.g. “The system noted severe headache with confusion, which often indicates possible stroke” (something comprehensible, whether from rule or model). Techniques like **model cards** and **explanation tools** will be part of our deployment ³⁴ ⁵⁰. We’ll generate a model card describing intended use, performance, and limitations, to be reviewed by clinical safety officers. We also maintain a **hazard log** – each known failure mode (e.g. confusion with negations in text) and how we mitigated it, and track if any new ones occur post-deployment (which then get added and addressed).

6. Continuous Improvement Loop: Deployment is not one-and-done. We will use a **feedback loop** where clinician overrides and patient outcomes feed back into model updates ⁵⁰. For instance, if clinicians frequently mark a certain symptom as red flag that the model was ignoring, we update either the lexicon or retrain the model to include that pattern. We might even implement a mechanism for clinicians to easily report “the triage suggestion was inappropriate for this case” via a button, which flags the case in our data for review. This human feedback is invaluable to correct the system. Additionally, periodic re-training (with latest data) will be scheduled, but only after validation steps each time (per the MLOps lifecycle in our plan: train, evaluate safety, canary deploy, then full deploy ³⁴). Essentially, every model update goes through the same rigorous validation as the first one.

7. Regulatory Compliance in Deployment: We will also integrate guardrails related to **consent and scope**: for example, ensuring that if a patient had opted out of AI triage (some systems allow opt-out), then for that patient we bypass ML and use manual triage. Or if our model is certified for adult patients, have a rule that if patient age <18, don’t use the model (since peds might not have been in training data). These kinds of scope guardrails prevent the model from being used outside its intended domain.

8. Vendor/Tool Integration Considerations: If any part of the system uses an external vendor (say an NLP API), additional guardrails apply: e.g. **no PHI is sent without encryption and a BAA**, and if that service fails, we default to internal logic. Since we lean OSS/self-hosted, we likely avoid runtime calls to third-party. But if we use, say, an external address validation for location extraction, we ensure that’s done on de-identified data or via a HIPAA-compliant route.

Finally, to illustrate how these guardrails work together, consider a scenario: The model says “likely not emergency”, rules say “no red flags”, but the narrative is slightly unusual. The LLM second-check says “I have low confidence here”. Guardrails kick in: the case is flagged for human review (a GP sees it in 5 minutes). Meanwhile, the patient gets a standard safe advice message with a safety-netting line: “If symptoms worsen, contact 111 or 999” (we always include that anyway ⁵⁶). The human reviews and either upgrades it or confirms it’s fine. If the model truly missed something, we log that miss, add that scenario to training. If everything was fine, no harm done – slight delay for that patient perhaps, but better safe than sorry. Contrast another scenario: Model or LLM has a glitch and starts labeling everything as safe including “chest pain” cases. Immediately the rule override for “chest pain” still diverts those, an alert “model disagreeing with rule 10 times today” pops up, and we perhaps auto-disable the model until we fix the bug. These examples show how a layered defense (rules, human, monitoring) prevents catastrophic failures.

In conclusion, the deployment of a mixed ML + rules system will be accompanied by a **robust safety net**: deterministic overrides for known risks, careful thresholding and human oversight for uncertain cases, comprehensive monitoring for any performance deviances, and the ability to fall back to a simpler system at any sign of trouble ⁵⁷. This approach aligns with both regulatory expectations and industry best practices for clinical AI (often summarized as “*human in the loop, AI as an adjunct*”). By following this plan, we aim to incrementally **upgrade our triage system’s intelligence while**

maintaining the same level of safety (or better) that our patients and clinicians currently expect, and always being prepared to roll back if needed ³⁴.

Roadmap for Incremental Implementation

Finally, we outline a phased **roadmap** to implement these enhancements, ensuring each step is evaluated before moving to the next:

Phase 1: Data Preparation and Baseline Improvements

- **1A. Dataset Curation:** Aggregate the past 6-12 months of triage submissions and outcomes into the structured dataset as defined (narratives, features, labels). Engage clinicians to help retrospectively label any ambiguous cases or confirm overrides so our ground truth is as accurate as possible. Perform de-identification on text and establish the secure data pipeline. This phase results in a “triage training dataset v1” ready for modeling.
- **1B. Lexicon and Rules Audit:** In parallel, review and update the current red-flag lexicon with clinical stakeholders. Low-hanging fruit: incorporate any obvious missing keywords and remove any that caused frequent false alarms (to improve precision a bit). This is a quick win to improve the deterministic side while data is being prepared.
- **1C. Train Initial ML Models:** Use the curated data to train initial versions of the emergency classifier and/or priority model (likely starting with something like a fine-tuned ClinicalBERT or a gradient boosted trees model on text features). Ensure this model is *not* deployed yet – just evaluate offline. Aim to see if its potential recall/precision tradeoff could beat our current system’s performance (which we can estimate from historical data: e.g. current rules might have 100% recall, X% precision on emergencies). If the model can reach 100% recall with better precision, that’s promising. We also train a secondary model for triage priority (or combine into one model with multi-class outputs). Evaluate these on holdout data and document the metrics.

Phase 2: Shadow Evaluation and Model Tuning

- **2A. Shadow Mode Launch:** Integrate the ML model into the triage pipeline in “silent” mode. It receives inputs and makes predictions but does not affect patient routing. We log its decisions alongside actual outcomes for a period (say 1-2 months, covering a reasonable volume and variety).
- **2B. Collect Results & Compare:** Analyze the shadow data. Key outputs: confusion matrix of model vs actual decisions, list of false negatives (if any), and list of false positives. For each discrepancy, perform a *case review* with clinicians. This is essentially an audit: did the model miss something obvious? Are its false positives actually acceptable (maybe our current system was too cautious or not cautious enough)? This step may reveal needed adjustments – e.g. we might discover the model missed all cases that mention “garbled speech” because our training data labeled them safe (maybe none were strokes in training), so we add “garbled speech” to red flags *and* retrain the model with those labeled as emergency. Iteratively refine the model with this feedback.
- **2C. Performance Checkpoint:** Decide if the model is ready for limited deployment. Criteria: *zero* false negatives in shadow test (or an extremely small number reviewed as non-critical with mitigations in place), and a false positive rate that stakeholders agree is manageable. Also confirm latency meets requirements in the live environment. If criteria not met, either further model improvement (additional training data, tweak architecture) or decide to keep it shadow until it improves. This phase might iterate a couple of times (train → shadow → adjust).

Phase 3: Limited Deployment with Overrides

- **3A. Canary Release:** Enable the model’s decisions for a small subset of real traffic (e.g. 5-10% of practices, or during out-of-hours only, or new patients only – any segmentation that limits risk exposure) ³⁴. The model now actively determines diversion vs continue, but with all guardrails on: red-

flag override still in effect, and perhaps we require clinician confirmation for any model-initiated diversions at this early stage. Essentially, in canary phase we treat each model diversion as a suggestion that a clinician quickly double-checks (we can't practically check them all if volume is high, but in 5% traffic, it might be fine). Alternatively, we deploy without requiring confirmation but closely monitor each case.

- **3B. Monitor Canary Outcomes:** After a defined trial period (say 2-4 weeks), evaluate: Did any emergency get missed in the canary group? How many were diverted and were those appropriate? Gather clinician feedback: are they noticing better triage or more work? Also check technical stability. If any serious issue arises (e.g. a missed emergency), we pause and address it before proceeding. If performance is as expected (e.g. model diverted maybe 3% of cases and all genuinely seemed urgent, precision improved, etc.), then we gain confidence. This phase is essentially an **early user acceptance test** in real setting.

- **3C. Gradual Rollout:** Increase the deployment scope gradually, e.g. 30% → 50% → 100% over several weeks, each time ensuring metrics remain stable. We'll use deployment flags to quickly revert to old system if needed. During this time, maintain a high level of logging and alerting. Perhaps keep the shadow mode running on the rest of traffic to continuously compare (until full rollout).

Phase 4: Integrate LLM Assist (Pilot)

(This phase is optional and only after Phase 3 is stable.)

- **4A. Pilot LLM Feature Extraction:** Choose one assistive use of LLM to pilot – for instance, use an internal medium-sized LLM to generate a list of symptoms from the narrative. We'd run this in shadow first on live data to see if it provides useful consistency. If it does, integrate it such that the output feeds the model/rules. Monitor its impact (does it catch something new? Does it ever mislead the model?). Start with a very constrained integration: e.g. only use LLM output for informational logging, then maybe actually influencing decisions if confident. This is a mini shadow→canary process on its own.

- **4B. "LLM Second-Check" Trial:** We can test the secondary review concept: route certain cases (maybe the model-safe but medium-risk ones) to a clinician with an LLM-generated note saying why it might be risky. Measure if this changes outcomes (do clinicians upgrade those more often or is it wasted?). This would be a careful study and might require user design (to not overwhelm clinicians). If positive, consider scaling up this approach.

- **4C. Evaluate LLM Value:** After piloting, decide if the LLM component is adding enough benefit relative to complexity. If yes, gradually widen its use; if not, it remains an experimental tool.

Phase 5: Full Deployment and Continuous Improvement

- **5A. Full System Deployment:** At this point, the new hybrid system (rules + ML + any LLM assist) is handling all intakes. We formally update our internal policies to reflect this (clinical risk management file, update user-facing documentation if needed to mention AI assistance, etc.). We ensure all staff are aware and trained on any new interfaces or procedures (e.g. if they now see AI-generated summaries).

- **5B. Ongoing Monitoring & Maintenance:** Establish a cadence for model retraining (maybe every 6 months, or whenever significant drift is detected or new data suggests a change). Each retraining follows a similar validation: offline tests, shadow (maybe shorter since we trust pipeline but still verify metrics), canary, then replace old model. Essentially, we institutionalize the MLOps cycle described in our algorithm spec ³⁴. We also maintain the rules – periodically review the red-flag list for updates (perhaps emerging medical guidance or seasonal issues). Regular audits (maybe quarterly) of a sample of cases to ensure no degradation.

- **5C. External Benchmarking & Certification:** As a longer-term goal, we might validate the system on external benchmarks (for academic publication or certification). For example, use a published set of triage scenarios to see how our system performs vs others, to demonstrate compliance with any future NHS AI standards or get CE marking if required. Additionally, engage with regulators (MHRA for software as a medical device, if applicable) to ensure our approach remains legally sound.

- **5D. Feedback Loop with Clients:** Since OneCare works with practices, we may gather feedback from

the clinicians using it – are they seeing improved triage (e.g. less unnecessary urgent alerts or fewer missed urgencies)? Use that to further tune thresholds or UX (maybe the priority categories can be adjusted if too many are marked urgent unnecessarily, etc.). Essentially, ensure the system is delivering practical value: faster triage, safe outcomes, and that stakeholders are satisfied.

This roadmap emphasizes **incremental upgrades**: at each step, we evaluate and only then proceed, which mitigates risk. By shadow testing and phased rollout, we fulfill the requirement of small-step changes that keep the system explainable and auditable at all times. Even if we envision eventually replacing a component (e.g. using ML instead of a simple rule for something), we will have gathered extensive evidence to justify that replacement, and we'll keep the old component as fallback until the new one is unquestionably better.

Throughout these phases, documentation will be kept up to date – not only for regulatory compliance but to have a knowledge base for the team (so that, for example, new members can see the evolution of the model and why certain decisions were made). We will also continuously refer back to **HIPAA, GDPR, and NHS guidelines** at each phase to ensure no step inadvertently violates privacy (for instance, before using patient data in a new way, double-checking consent, etc.) ²⁴ ⁴⁰. Any introduction of an external tool or change in data handling will trigger a review (DPIA update, etc.).

By following this plan, OneCare can evolve its safety gate and triage scoring into a **state-of-the-art, hybrid intelligent system**. It will leverage modern NLP/LLM capabilities to improve accuracy and efficiency, *while rigorously maintaining safety, speed, explainability, and compliance*. In summary, we will enhance our triage with data-driven methods grounded in evidence and careful validation, ensuring better outcomes for patients and manageable workflows for clinicians as we deploy these advanced technologies in urgent care triage.

Sources: The research and recommendations above are informed by a range of academic and industry sources, including studies on AI triage performance ⁴ ⁶, safety evaluations of LLMs in clinical settings ¹¹, best-practice frameworks for clinical AI deployment ³⁴ ⁵⁷, and official guidance on health data use and governance ²⁴ ⁴⁰. All cited works are listed in the reference citations throughout this document for further reading and verification.

¹ ² ³ ³¹ ³² ³³ ³⁴ ³⁵ ⁴⁶ ⁴⁹ ⁵⁰ ⁵³ ⁵⁴ ⁵⁵ ⁵⁶ ⁵⁷ Algorithm.md

<https://github.com/georgeahmed/onecare/blob/fa507f8f0e9f4e8693cb508f1a4d9387123017c0/Algorithm.md>

⁴ ⁵ ⁶ ⁷ ⁸ ¹⁶ ⁵¹ ⁵² A foundational triage system for improving accuracy in moderate acuity level emergency classifications | Communications Medicine

https://www.nature.com/articles/s43856-025-01052-w?error=cookies_not_supported&code=424f51ce-ce52-4105-a4ec-bfa8149413f2

⁹ Evaluating the Accuracy of AI Chatbots in Triageing Emergency Cases

[https://www.jem-journal.com/article/S0736-4679\(25\)00320-8/fulltext](https://www.jem-journal.com/article/S0736-4679(25)00320-8/fulltext)

¹⁰ ¹⁵ Investigating LLMs in Clinical Triage: Promising Capabilities ... - arXiv

<https://arxiv.org/html/2504.16273v1>

¹¹ Chat-GPT in triage: Still far from surpassing human expertise

<https://www.sciencedirect.com/science/article/abs/pii/S0735675725001937>

¹² ¹⁸ ¹⁹ ²⁰ C-PATH: Conversational Patient Assistance and Triage in Healthcare System

<https://arxiv.org/html/2506.06737v1>

13 Detecting emergencies in patient portal messages using large ...

<https://academic.oup.com/jamia/article-abstract/32/6/1032/8112816>

14 Search AMIA.org | AMIA - American Medical Informatics Association

<https://amia.org/search?>

[f%5B0%5D=type%3AMeetings&f%5B1%5D=type%3AOther&f%5B2%5D=ujournal%3AApplied%20Clinical%20Informatics&f%5B3%5D=ujournal%3AJ](https://academic.oup.com/jamia/article-abstract/32/6/1032/8112816)

17 A foundational triage system for improving accuracy in moderate ...

<https://www.nature.com/articles/s43856-025-01052-w>

21 22 23 24 44 45 Artificial Intelligence - NHS Transformation Directorate

<https://transform.england.nhs.uk/information-governance/guidance/artificial-intelligence/>

25 26 27 De-identification of patient notes with recurrent neural networks - PMC

<https://pmc.ncbi.nlm.nih.gov/articles/PMC7787254/>

28 Evaluating GPT models for clinical note de-identification - Nature

<https://www.nature.com/articles/s41598-025-86890-3>

29 De-identification of clinical notes via recurrent neural network and ...

<https://www.sciencedirect.com/science/article/pii/S1532046417301223>

30 pyDeid: an improved, fast, flexible, and generalizable rule-based ...

<https://academic.oup.com/jamiaopen/article/8/1/ooae152/7966802>

36 38 39 40 41 42 43 When AI Technology and HIPAA Collide

<https://www.hipaajournal.com/when-ai-technology-and-hipaa-collide/>

37 Does AI Comply with HIPAA? | Understanding the Key Rules

<https://www.hipaavault.com/resources/does-ai-comply-with-hipaa/>

47 Synthesizing Healthcare Data for AI, With HIPAA Expert Determination

<https://www.tonic.ai/guides/hipaa-ai-compliance>

48 De-identification is not enough: a comparison between de-identified ...

<https://www.nature.com/articles/s41598-024-81170-y>